

PART 1: CONNECT TO HADOOP SERVER

Step 1: Login

```
ssh hadoop@10.10.15.81
```

 Enter password:

```
hadoop
```

PART 2: CREATE YOUR WORKING DIRECTORY

Step 2: Check files

```
ls
```

Step 3: Create folder (Roll No)

```
mkdir 31415  
cd 31415
```

PART 3: CREATE INPUT FILE

Step 4: Create dataset file

```
nano sample_weather.txt
```

 Paste this:

```
2023-01-01,30,20,10  
2023-01-02,32,22,12  
2023-01-03,28,18,8  
2023-01-04,35,25,15  
2023-01-05,31,21,11
```

 Save:

- Ctrl + O → Enter
 - Ctrl + X
-

PART 4: CREATE JAVA PROGRAM

Step 5: Create Java file

```
nano WeatherAnalysis.java
```

👉 Paste your full code (already correct 👍)

👉 Save:

- Ctrl + O → Enter
 - Ctrl + X
-

🔷 PART 5: COMPILE PROGRAM

✅ Step 6: Compile Java

```
javac -classpath `hadoop classpath` -d . WeatherAnalysis.java
```

👉 What it does:

- Compiles Java file
 - `-d .` → creates proper package structure
-

🔷 PART 6: CREATE JAR FILE

✅ Step 7: Create JAR

```
jar -cvf weather.jar *.class
```

👉 Combines all `.class` files into executable JAR

🔷 PART 7: START HADOOP SERVICES

✅ Step 8: Start HDFS

```
start-dfs.sh
```

✅ Step 9: Start YARN

```
start-yarn.sh
```

✅ Step 10: Check processes

```
jps
```

👉 You must see:

- NameNode
 - DataNode
 - SecondaryNameNode
 - ResourceManager
 - NodeManager
-

! If NameNode missing:

```
hdfs namenode -format  
start-dfs.sh  
start-yarn.sh
```

◆ PART 8: HDFS SETUP (VERY IMPORTANT)

✓ Step 11: Remove old folder (if exists)

```
hdfs dfs -rm -rf /31415  
hdfs dfs -rm -rf /output
```

✓ Step 12: Create your folder

```
hdfs dfs -mkdir /31415
```

✓ Step 13: Create input directory

```
hdfs dfs -mkdir /31415/input
```

◆ PART 9: UPLOAD FILE TO HDFS

✓ Step 14: Upload dataset

```
hdfs dfs -put sample_weather.txt /31415/input
```

👉 This moves file → HDFS

◆ PART 10: RUN MAPREDUCE JOB

✓ Step 15: Execute job

```
hadoop jar weather.jar WeatherAnalysis /31415/input /31415/output
```

👉 Arguments:

- `/31415/input` → input folder
 - `/31415/output` → output folder
-

◆ PART 11: CHECK OUTPUT

✓ Step 16: List output

```
hdfs dfs -ls /31415/output
```

✓ Step 17: Display result

hdfs dfs -cat /31415/output/part-r-00000



EXPECTED OUTPUT

DewPoint 21.2
Temperature 31.2
WindSpeed 11.2



WHAT YOUR PROGRAM IS DOING

◆ Mapper:

For each row:

2023-01-01,30,20,10



Output:

Temperature → 30
DewPoint → 20
WindSpeed → 10

◆ Shuffle:

Temperature → [30,32,28,35,31]
DewPoint → [20,22,18,25,21]
WindSpeed → [10,12,8,15,11]

◆ Reducer:

Temperature avg = 31.2
DewPoint avg = 21.2
WindSpeed avg = 11.2

CODE:

WeatherAnalysis.java

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
```

```

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WeatherAnalysis {

    // ♦ Mapper Class
    public static class WeatherMapper
        extends Mapper<Object, Text, Text, DoubleWritable> {

        private Text type = new Text();
        private DoubleWritable value = new DoubleWritable();

        public void map(Object key, Text line, Context context)
            throws IOException, InterruptedException {

            String[] fields = line.toString().split(",");

            // Safety check (avoid ArrayIndexOutOfBoundsException)
            if (fields.length < 4) return;

            double temp = Double.parseDouble(fields[1]);
            double dew = Double.parseDouble(fields[2]);
            double wind = Double.parseDouble(fields[3]);

            // Emit Temperature
            type.set("Temperature");
            value.set(temp);
            context.write(type, value);

            // Emit Dew Point
            type.set("DewPoint");
            value.set(dew);
            context.write(type, value);

            // Emit Wind Speed
            type.set("WindSpeed");
            value.set(wind);
            context.write(type, value);
        }
    }

    // ♦ Reducer Class
    public static class WeatherReducer
        extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

        public void reduce(Text key, Iterable<DoubleWritable> values,
            Context context)
            throws IOException, InterruptedException {

            double sum = 0;
            int count = 0;

            for (DoubleWritable val : values) {
                sum += val.get();
                count++;
            }

            double avg = sum / count;

```

```
        context.write(key, new DoubleWritable(avg));
    }
}

// ♦ Main Method
public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "Weather Analysis");

    job.setJarByClass(WeatherAnalysis.class);

    job.setMapperClass(WeatherMapper.class);
    job.setReducerClass(WeatherReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);

    // Input and Output Path
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

sample_weather.txt

```
2023-01-01,30,20,10
2023-01-02,32,22,12
2023-01-03,28,18,8
2023-01-04,35,25,15
2023-01-05,31,21,11
```